

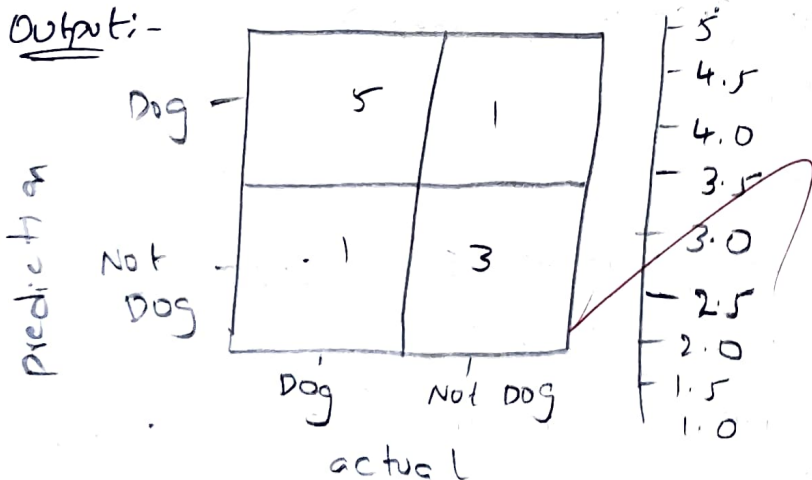
Experiment -1

1) Aim:- To demonstrate confusion matrix using Python.

Program:-

```
import numpy as np
import matplotlib.pyplot as plt
actual = np.array(['dog', 'dog', 'dog', 'Not dog'])
predicted = np.array(['dog', 'Not dog', 'dog'])
conf_matrix = ConfusionMatrix(actual, predicted)
sns.heatmap(ConfMatrix, annot=True, fmt='g',
            xticklabels=['dog', 'Not dog'],
            yticklabels=['dog', 'Not dog'], cmap='RdPo')
plt.ylabel("prediction", fontsize=16)
plt.xlabel("actual", fontsize=16)
plt.title("Confusion matrix", fontsize=20)
plt.show()
```

Output:-



Result:- The program was executed successfully.

Experiment - 2

2) Aim:- To demonstrate 2 class confusion matrix using python

Program:-

```
import pandas as pd
import matplotlib.pyplot as plt import seaborn
as sns
from sklearn.model_selection
import train-test-split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score,
precision_score, recall, confusion_matrix
sns.heatmap (conf-matrix, annot = True, fmt = 'd',
cmap = 'PuBuGn', xticklabels = wine
target-names [:2], yticklabels = wine.target ([:2])
plt.xlabel ("predicted label")
plt.ylabel ("true label")
plt.title ("Confusion-matrix")
plt.show()
```

Output:-

accuracy = 0.9743589743589

Classification Report:-

	precision	recall	f1-score	support
class 0	1.00	0.94	0.97	17
class-1	0.96	1.00	0.98	22

Result:- The program was executed successfully.

Experiment - 3

3) Aim:- To analyse the performance of a multi class.

program:-

```
from sklearn.datasets import load_digits
from sklearn.model_selection
import train_test_split from sklearn.ensemble
import RandomForestClassifier
from sklearn.metrics import sklearn.metrics
X, y = load_digits (return_X_y = True).
X_train, X_test, y_train, y_test =
train_test_split (X, y, test = 0.25)
clf = RandomForestClassifier. (random_state = 23)
clf. fit (X_train, y_train)
Matrix', fontsize = 17)
plt.show()
accuracy_score (y_test, "y_pred")
print ("Accuracy: ", accuracy)
```

Output:-

0	49	0	0	0	0	0	0	0	0	50
1	0	38	0	0	0	0	0	0	0	40
2	0	0	41	0	0	0	0	0	0	30
3	0	0	0	43	0	0	0	0	0	20
4	0	0	0	0	34	0	0	0	0	10
5	0	0	0	0	0	34	0	0	0	10
6	0	0	0	0	0	0	10	0	0	10
7	0	0	0	0	0	0	0	10	0	10
8	0	0	0	0	0	0	0	0	10	10
9	0	0	0	0	0	0	0	0	0	10
	0	1	2	3	4	5	6	7	8	9

Accuracy = 0.977777

Result:- The program was executed successfully.

Experiment - 4

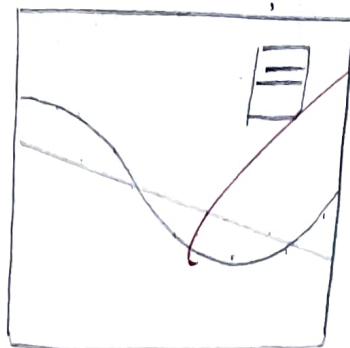
- 4) Aim:- To analyse the performance of a Over fitting by using choosen database with python code.

Py Program 1:-

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
X-test = np.linspace(0, 1, 100)
plt.plot(X-test, pipeline.predict(X-test,
    [:, np.newaxis])), label="Model")
plt.plot(X-test, true_fun(X-test),
    label="True function")
plt.scatter(X, y, edgecolor="y",
    s=20, marker="x",
    label="Samples")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Degree 1

Output:-



Result:-

The program was executed successfully.

Experiment - 5

5) Aim:- To demonstrate the performance of a linear regression by using chosen database with python code.

Program:-

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

plt.figure(figsize=(10,6))
plt.ylabel('sepal width (cm)')
plt.title('Linear Regression : Sepal width vs  
sepal length')
plt.legend()
plt.show()

new-sample = pd.DataFrame([5])
columns = ['sepal length (cm)']
```

Output:-

Mean squared Error on test set: 0.23

the predicted output for sepal length [[5]]

Linear Regression : sepal width vs sepal length

Result:-

The program was executed successfully.